

TEORÍA DE ALGORITMOS
(75.29) CURSO BUCHWALD - GENENDER

Trabajo Práctico 3

Diversión NP-Completa

17 de diciembre de 2024

Benjamin Castellano
Bogdan
111519

Joaquin Eduardo
Embon
111292

Lionel Gabriel
Maydana Gonzalez
106512

1. Introduccion

En este trabajo vamos a estar haciendo un profundo analisis sobre el nuevo juego al que estan jugando Sophia y Mateo: La Batalla Naval Individual.

En dicho juego, tenemos un tablero de $n \times m$ casilleros, y k barcos. Cada barco i tiene b_i de largo. Es decir, requiere de b_i casilleros para ser ubicado. Todos los barcos tienen 1 casillero de ancho. El tablero a su vez tiene un requisito de consumo tanto en sus filas como en sus columnas. Si en una fila indica un 3, significa que deben haber 3 casilleros de dicha fila siendo ocupados. Ni más, ni menos. No podemos poner dos barcos de forma adyacente (es decir, no pueden estar contiguos ni por fila, ni por columna, ni en diagonal directamente). Debemos ubicar todos los barcos de tal manera que se cumplan todos los requisitos.

Vamos a estar analizandolo de distintas formas. Vamos a proponer soluciones para este problema, mediante un algoritmo de backtracking y por otro lado un modelo de programacion lineal, para los cual vamos a mostrar sus respectivos analisis de complejidad y haciendo mediciones para cada uno. Luego vamos a proponer distintas aproximaciones polinomiales para este problema, primero mediante un aproximación propuesta por la cátedra, y luego vamos a dar un algoritmo greedy que aproxime una solucion para este problema. Para cada aproximación vamos a demostrar y descubrir la calidad de estas aproximaciones. Como último estudio de este problema, previo a las soluciones nombradas anteriormente, vamos a demostrar que este problema en su version de decision, esta en NP, y particularmente, es un problema NP-Completo.

2. NP-Completo

2.1. ¿Está en NP?

Para comenzar a demostrar que el problema de la Batalla Naval Individual es NP-Completo, primero tenemos que demostrar que el problema está en NP (existe un validador de este problema con complejidad polinomial).

Vamos a enunciar el problema en su version de decisión:

Dado un tablero de $n \times m$ casilleros, y una lista de k barcos (donde el barco i tiene b_i de largo), una lista de restricciones para las filas (donde la restricción j corresponde a la cantidad de casilleros a ser ocupados en la fila j) y una lista de restricciones para las columnas (símil filas, pero para columnas), ¿es posible definir una ubicación de dichos barcos de tal forma que se cumplan con las demandas de cada fila y columna, y las restricciones de ubicación?

```
1 def verificador_eficiente(tablero_solucion, barcos, demandas_filas,
2   demandas_columnas):
3     """
4     Pre: Recibe:
5     - Un tablero con los barcos ubicados como solucion (siendo 0 espacios libres y
6       numeros los barcos).
7     - Lista de barcos original (cada indice i deberia ser el largo del barco i+1).
8     - Lista de demandas de filas original.
9     - Lista de demandas de columnas original.
10
11     Debemos validar que la solucion:
12     - Cubra todas las demandas de filas y columnas (ni mas ni menos).
13     - Los barcos sean los mismos que los de la lista original de barcos.
14     - Los barcos cumplan con la ley de adyacencia.
15     - No hayan barcos de mas.
16
17     Pos:
18     - Devuelve True en caso de que sea una solucion valida y False en caso
19       contrario.
20     """
21
22     demanda_restante = calcular_demandas_restantes(tablero_solucion, demandas_filas,
23     demandas_columnas)
24     if demanda_restante != 0:
25         return False
26
27     n, m = tablero_solucion.shape
28     barcos_checkeados = set()
29
30     for fila in range(len(tablero_solucion)):
31         for columna in range(len(tablero_solucion[0])):
32             if tablero_solucion[fila][columna] != 0 and not tablero_solucion[fila][
33             columna] in barcos_checkeados:
34                 barco = tablero_solucion[fila][columna]
35
36                 if barco < len(barcos):
37                     # Si esta en el rango de barcos, sera el largo que deberia
38                     tener originalmente.
39                     barco_largo = barcos[barco-1]
40                 else:
41                     return False
42
43                 if barco > 1:
44                     # Si el barco no es de un solo bloque de largo.
45                     if columna < m and tablero_solucion[fila][columna + 1] == barco
46                     :
47                         if not se_puede_colocar(tablero_solucion, barco,
48                         barco_largo, fila, columna, "H"):
49                             return False
50
51                     if fila < n and tablero_solucion[fila + 1][columna] == barco:
52                         if not se_puede_colocar(tablero_solucion, barco,
53                         barco_largo, fila, columna, "V"):
```

```
45         return False
46
47     else:
48         if not se_puede_colocar(tablero_solucion, barco, barco_largo,
49                                fila, columna, None):
50             return False
51
52         barcos_checkeados.add(barco)
53
54     return True
55
56 def calcular_demandas_restantes(tablero, demandas_filas, demandas_columnas):
57     """
58     Calcula cuanta demanda falta por cubrir en cada fila y columna.
59     """
60     demandas_restantes_filas = demandas_filas.copy()
61     demandas_restantes_columnas = demandas_columnas.copy()
62
63     for fila in range(len(tablero)):
64         for columna in range(len(tablero[0])):
65             if tablero[fila][columna] != 0: # Hay un barco en esta celda.
66                 demandas_restantes_filas[fila] = demandas_restantes_filas[fila] - 1
67                 demandas_restantes_columnas[columna] = demandas_restantes_columnas[
68                     columna] - 1
69
70     return demandas_restantes_filas, demandas_restantes_columnas
71
72 def se_puede_colocar(tablero, barco, barco_largo, fila, columna, orientacion):
73
74     filas, columnas = tablero.shape
75
76     if orientacion == 'H': # Horizontal.
77         # Verificar hacia la derecha.
78         if columna + barco_largo <= columnas:
79             if all(tablero[fila, columna:columna + barco_largo] == barco):
80                 # Verificar largo.
81                 if all(tablero[max(0, fila - 1):min(filas, fila + 2), max(0,
82                     columna - 1):min(columnas, columna + barco + 1)].flatten() == 0):
83                     # Verificar adyacencia. (Si el largo es mayor al que debería el
84                     barco, falla acá).
85                     return True
86
87     elif orientacion == 'V': # Vertical.
88         # Verificar hacia abajo.
89         if fila + barco_largo <= filas:
90             if all(tablero[fila:fila + barco_largo, columna] == barco):
91                 # Verificar largo.
92                 if all(tablero[max(0, fila - 1):min(filas, fila + barco_largo + 1),
93                     max(0, columna - 1):min(columnas, columna + 2)].flatten() == 0):
94                     # Verificar adyacencia.
95                     return True
96
97     elif not orientacion:
98         if all(tablero[max(0, fila - 1):min(filas, fila + 2), max(0, columna - 1):
99             min(columnas, columna + 2)].flatten() == 0):
100             # Verificar adyacencia.
101             return True
102
103     return False
```

Complejidad

El programa tiene tres funciones principales:

- **verificador_eficiente**: Revisa la validez de la solución según las restricciones dadas.
- **calcular_demandas_restantes**: Calcula las demandas restantes de filas y columnas.

- **se_puede_colocar**: Verifica si un barco cumple las restricciones de ubicación, largo y adyacencia.

El flujo principal ocurre en **verificador_eficiente**, donde se iteran las celdas del tablero y se validan las restricciones de cada barco.

verificador_eficiente (pasos principales)

- Entrada:
 - **tablero_solucion**: Matriz de tamaño $n \times m$.
 - **barcos**: Lista de longitud k , donde k es el número de barcos.
 - **demandas_filas** y **demandas_columnas**: Listas de tamaño n y m respectivamente.
- Cálculo de demandas restantes por medio la función **calcular_demandas_restantes**, la cual recorre cada celda del tablero realizando operaciones $\mathcal{O}(1)$, por lo que este paso sería $\mathcal{O}(n \times m)$, siendo n cantidad de filas y m cantidad de columnas del tablero.
- Recorremos todo el tablero $\mathcal{O}(n \times m)$. Por cada celda, verifica si contiene un barco y si no ha sido revisado previamente. Si encuentra un barco, realiza lo siguiente:
 - Determinación del largo del barco $\mathcal{O}(b_i)$: Se verifica que este dentro de la lista (que no hayan barcos de mas) y luego se le asigna el largo que le corresponde. Si no tiene ese largo, se detecta mas adelante ya sea corroborando el largo o la adyacencia.
 - Verificación de adyacencia a través de la función **se_puede_colocar** con la orientación correspondiente. En el peor caso realiza $\mathcal{O}(b_i)$ operaciones para corroborar el largo y $\mathcal{O}(b_i^2)$ para corroborar la adyacencia.
- La complejidad de esta sección queda $\mathcal{O}(n \times m \times b_i^2)$

calcular_demandas_restantes

- Esta función recorre cada celda del tablero para actualizar las demandas de filas y columnas. Su complejidad es $\mathcal{O}(n \times m)$.

se_puede_colocar

- Verifica si el largo del barco es el correcto en una orientación (horizontal, vertical o aislada).
- Evalúa adyacencia considerando submatrices de tamaño proporcional al largo del barco b_i .
 - Esto implica $\mathcal{O}(b_i^2)$ operaciones para cada barco.

Complejidad total

- En el peor de los casos todos los barcos son grandes y están en el tablero, se cumple con las demandas de las filas y columnas, nos quedaría una complejidad de: $\mathcal{O}(n \times m \times b_i^2)$.
- Al ser un problema con un verificador de complejidad polinomial, podemos afirmar que está en NP.

2.2. Demostración NP-Complejidad

Para demostrar que el problema de la Batalla Naval Individual es NP-Completo, vamos a utilizar el problema de Bin Packing en su versión unaria, que es NP-Completo. El problema de Bin Packing en su versión de decisión dice:

Dados un conjunto de números, cada uno expresado en código unario, una cantidad de bins B , expresada en código unario, y la capacidad de cada bin C , expresada en código unario; donde la suma del conjunto de números es igual a $C \times B$.

Pregunta: ¿Es posible asignar todos los objetos a los K contenedores de tal manera que la suma de elementos de cada contenedor es exactamente igual a la capacidad C ?

Dicho esto, y sabiendo que este problema es NP-Completo, nuestro objetivo es reducir Bin Packing polinomialmente a Batalla Naval para demostrar que este último es NP-Completo.

Proponemos la siguiente reducción:

Dado un conjunto de objetos $O = \{o_1, o_2, \dots, o_n\}$ con tamaños $s(o_i) \in \mathbb{N}$, un número K de contenedores disponibles y una capacidad fija C para cada contenedor, existe solución para el problema de Bin Packing, si y solo si, existe una disposición válida para un conjunto de barcos $A = \{a_1, a_2, \dots, a_n\}$ con tamaños $b(a_i) \in \mathbb{N}$ en un tablero de $2K - 1$ filas $\times$ ($C +$ cantidad de barcos) columnas, y unas listas de restricciones F y G para filas y columnas respectivamente.

Vamos a explicarlo con un ejemplo:

Consideramos el conjunto de objetos $O = 11111, 1111, 1111, 111, 1$, un número de contenedores $K = 2$, y una capacidad $C = 10$.

La cantidad y el tamaño de los barcos A se corresponde con la cantidad y tamaño de los objetos O . entonces tendremos un conjunto de barcos $A = 11111, 1111, 1111, 111$

Para las filas, armamos el tablero con una cantidad de $2K - 1$, ya que K filas corresponden a los K contenedores, y otras $K - 1$ filas se encuentran intercaladas entre las anteriores, y su objetivo es separar estas filas que representan los contenedores para que los barcos puedan ser ubicados, sin la molestia de que los barcos no pueden estar pegados (restricción del problema de la Batalla Naval). La cantidad de columnas tiene un objetivo similar, C es porque es la capacidad que los contenedores tienen, y a este valor le agregamos el número de barcos a ubicar. Agregamos esta cantidad para que los barcos puedan ser ubicados en los contenedores, sin que tengan la molestia de que los barcos no pueden estar pegados, pero en este caso dentro de cada contenedor. Para esta instancia del problema nos quedaría entonces filas $= 2 \cdot 2 - 1 = 3$ y columnas $= 10 + 4 = 14$:

Con respecto a las restricciones del tablero, la lista de restricciones de cada fila nos dice que para cada fila que represente a los contenedores, su capacidad va a ser igual a C (esto es porque lo que nos pide el problema de Bin Packing es que la suma de los elementos de cada contenedor sea exactamente igual a C), y las filas que creamos para que sean separadoras de las filas que representan los contenedores, van a tener una capacidad de 0, ya que no queremos meter barcos en estas filas.

10														
0														
10														

Por último, para armar la lista de restricciones de cada columna vamos a tener en cuenta las longitudes de los barcos. Antes de definir las, las capacidades comienzan en 0.

	0	0	0	0	0	0	0	0	0	0	0	0	0	0
10														
0														
10														

Vamos a ir incrementándolas de la siguiente manera: vamos a recorrer el arreglo de barcos, que va a estar ordenado por tamaño, de mayor a menor. Por cada barco vamos a ir incrementando desde la columna i hasta la columna $i +$ longitud del barco. Entonces para el primer barco (11111) incrementaríamos las primeras 6 (desde la 0 a la 5) columnas. Cabe aclarar que los barcos que

vayamos utilizando, no serán considerados para próximas iteraciones del arreglo, así que el arreglo de barcos quedaría: (11111, 11111, 1111). Y la grilla:

	1	1	1	1	1	1	0	0	0	0	0	0	0
10													
0													
10													

Antes de considerar el siguiente barco, vamos a saltar una columna con el objetivo de respetar el requerimiento que dice que los barcos no pueden estar juntos, así que para el próximo barco, comenzaremos a incrementar desde la columna 7 en adelante (saltamos la columna 6). Lo último que vamos a considerar para terminar de definir las restricciones de las columnas, es la suma acumulada de espacio ocupado, es decir, en este caso por ejemplo, llevaríamos ocupado un espacio de 6, así que no podríamos agregar el próximo barco en el arreglo de barcos (el barco 11111 tiene longitud 5), porque sino estaríamos superando la capacidad $C = 10$ para esta iteración, así que seguiremos buscando en el arreglo hasta encontrar el próximo barco que podamos agregar. En este caso va a ser el 1111, así que la grilla quedaría:

	1	1	1	1	1	1	0	1	1	1	1	1	0
10													
0													
10													

Si llegamos a esta instancia, en la que el valor acumulado de barcos para esta iteración del arreglo es igual a 10, o no encontramos otro barco que podamos agregar, comenzamos a iterar el arreglo desde el principio, dejamos de tener en cuenta el valor acumulado anterior, y comenzamos a incrementar nuevamente desde la primera columna. Nuestro arreglo actualmente es (11111, 11111), así que tomamos el primero. Nos queda la grilla:

	2	2	2	2	2	1	0	1	1	1	1	1	0
10													
0													
10													

Por último, agregamos el 11111 restante:

	2	2	2	2	2	1	1	2	2	2	2	1	0
10													
0													
10													

Así ya quedaría definida la grilla. Con estos valores, el problema de la batalla naval encuentra la solución:

	2	2	2	2	2	1	1	2	2	2	2	1	0
10	x	x	x	x	x	x		x	x	x	x		
0													
10	x	x	x	x	x	x	x	x	x	x	x		

Esto devuelve true, la solución correcta para el problema de Bin Packing, resuelta por el problema de la batalla naval.

Dejamos claro como es el funcionamiento de nuestra reducción, pero tenemos que responder dos preguntas: ¿Es una reducción polinomial? ¿Esto funciona siempre?.

Primero esta es una reducción polinomial porque:

- El conjunto de barcos se corresponde con el conjunto original del problema de Bin Packing.
- Creamos la matriz en tiempo La complejidad es $\mathcal{O}((2K - 1) * (C + N))$ siendo N la cantidad de barcos
- Definimos las restricciones de cada fila en $\mathcal{O}(Cant.filas)$ y para definir las restricciones de cada columna, recorremos nuestro arreglo de barcos, a la vez que incrementamos las capacidades de nuestras columnas. En el peor de los casos, podríamos llegar a recorrer el arreglo de barcos por cada elemento de este, así que la complejidad de este punto nos quedaría $\mathcal{O}(n^2 + Cant.columnas)$

Uno podría llegar a pensar a esta reducción como pseudo-polinomial, por las complejidades señaladas anteriormente, que están definidas por los parámetros del problema original, pero no hay que olvidar que estamos considerando el problema de Bin Packing en su versión unaria, por lo que los parámetros crecen polinómicamente respecto del tamaño de la entrada, así que podemos concluir que esta reducción es polinomial.

Para concluir vamos a demostrar que esta reducción es correcta (el si solo si):

Podemos asegurar que hay solución a Bin Packing si hay una disposición posible de barcos en la Batalla Naval ya que creamos un tablero que representa exactamente una instancia del problema de Bin Packing, eligiendo restricciones para las filas que corresponden a la capacidad de cada contenedor, y restricciones de columnas que permitan encontrar una solución sin preocuparnos por las restricciones extra de ubicación que el problema de la Batalla Naval contiene, permitiendo que el algoritmo trabaje sobre una instancia totalmente adaptada a Bin Packing.

Además podemos asegurar que hay solución a la Batalla Naval si hay solución para Bin Packing, ya que, estas instancias de la Batalla Naval están adaptadas a Bin Packing, entonces si pensamos a las filas como contenedores, a la capacidad de cada contenedor como la cantidad de columnas - cantidad de barcos, y a los barcos como un conjunto de números, iguales a la longitud de cada barco, el problema de Bin Packing va a devolver true en caso de encontrar una solución válida, que podemos asegurar que existe en el problema de la Batalla Naval.

Con esto demostramos que la Batalla Naval, es un problema NP-Completo.

3. Algoritmo Backtracking

Para las consignas de ahora en adelante, vamos a tener en cuenta la consigna en la versión de optimización: Dado un tablero de $n \times m$ casilleros, y una lista de k barcos (donde el barco i tiene b_i de largo) una lista de las demandas de las n filas y una lista de las m demandas de las columnas, dar la asignación de posiciones de los barcos de tal forma que se reduzca al mínimo la cantidad de demanda incumplida. Pueden no utilizarse todos los barcos. Si simplemente no se cumple que una columna que debería tener 3 casilleros ocupados tiene 1, entonces contará como 2 de demanda incumplida. Por el contrario, no está permitido exceder la cantidad demandada.

Se implementó un algoritmo de backtracking para hallar el óptimo de demanda cumplida en cada posible caso del problema de batalla naval. El resultado en código de esta implementación se encuentra debajo.

```
1 import numpy as np
2
3 def puede_colocar_barco(tablero, barco, fila, columna, orientacion, demandas_filas,
4   demandas_columnas):
5     """
6     Verifica si es posible colocar un barco hacia abajo si es con orientacion "
7     vertical" o hacia
8     derecha si es con orientacion "horizontal" respetando las restricciones de:
9     - Adyacencia.
10    - Demandas de filas y columnas.
11    """
12    filas, columnas = tablero.shape
```

```
12     if orientacion == 'H': # Horizontal
13         # Verificar hacia la derecha
14         if columna + barco <= columnas:
15             if all(tablero[fila, columna:columna + barco] == 0) and \
16                 demandas_filas[fila] - barco >= 0 and \
17                 all(demandas_columnas[j] - 1 >= 0 for j in range(columna,
columna + barco)):
18                 # Verificar adyacencia
19                 if all(tablero[max(0, fila - 1):min(filas, fila + 2), max(0,
columna - 1):min(columnas, columna + barco + 1)].flatten() == 0):
20                     return True
21
22     elif orientacion == 'V': # Vertical
23         # Verificar hacia abajo
24         if fila + barco <= filas:
25             if all(tablero[fila:fila + barco, columna] == 0) and \
26                 demandas_columnas[columna] - barco >= 0 and \
27                 all(demandas_filas[i] - 1 >= 0 for i in range(fila, fila +
barco)):
28                 # Verificar adyacencia
29                 if all(tablero[max(0, fila - 1):min(filas, fila + barco + 1), max
(0, columna - 1):min(columnas, columna + 2)].flatten() == 0):
30                     return True
31
32     return False
33
34
35 def colocar_barco(tablero, barco, fila, columna, orientacion, id_barco,
demandas_fila, demandas_columna):
36     """
37     Coloca o retira un barco en el tablero, eligiendo la direccion y actualiza
demandas.
38     - Si es horizontal ('H'), elige extenderse a la derecha.
39     - Si es vertical ('V'), elige extenderse hacia abajo.
40     """
41     if orientacion == 'H':
42         for i in range(barco):
43             tablero[fila][columna + i] = id_barco
44             if id_barco != 0:
45                 demandas_columna[columna + i] -= 1
46             else:
47                 demandas_columna[columna + i] += 1
48         if id_barco != 0:
49             demandas_fila[fila] -= barco
50         else:
51             demandas_fila[fila] += barco
52
53     elif orientacion == 'V':
54         for i in range(barco):
55             tablero[fila + i][columna] = id_barco
56             if id_barco != 0:
57                 demandas_fila[fila + i] -= 1
58             else:
59                 demandas_fila[fila + i] += 1
60         if id_barco != 0:
61             demandas_columna[columna] -= barco
62         else:
63             demandas_columna[columna] += barco
64
65
66 def obtener_filas_columnas_prioritarias(demandas_filas, demandas_columnas):
67     """
68     Obtiene listas de filas y columnas priorizadas por demanda restante absoluta (
mayor urgencia).
69     """
70     # Priorizar por la demanda restante absoluta.
71     filas_prioritarias = np.argsort(-demandas_filas).tolist() # Devuelve los
indices.
72     columnas_prioritarias = np.argsort(-demandas_columnas).tolist()
73
```

```

74     return filas_prioritarias, columnas_prioritarias
75
76
77 def batalla_naual(tablero, barcos, demandas_filas, demandas_columnas):
78     # Ordenar los barcos de mayor a menor longitud.
79     barcos = sorted(barcos, reverse=True)
80     demanda_total = demandas_filas.sum() + demandas_columnas.sum()
81     sol_optima = [demanda_total, tablero.copy(), 0]
82     sol_parcial = [demanda_total, tablero.copy(), 0] #[demanda incumplida, tablero,
83         ultimo barco usado]
84
85     batalla_naual_bt(tablero, barcos, demandas_filas, demandas_columnas, 0,
86         sol_parcial, sol_optima)
87     return demanda_total - sol_optima[0], sol_optima[1]
88
89 def batalla_naual_bt(tablero, barcos, demandas_filas, demandas_columnas, indice,
90     sol_parcial, sol_optima):
91
92     if indice >= len(barcos):
93         if sol_parcial[0] < sol_optima[0]:
94             sol_optima[0] = sol_parcial[0]
95             sol_optima[1] = sol_parcial[1].copy()
96             sol_optima[2] = sol_parcial[2]
97         return
98
99     if sol_parcial[0] < sol_optima[0]:
100         sol_optima[0] = sol_parcial[0]
101         sol_optima[1] = sol_parcial[1].copy()
102         sol_optima[2] = sol_parcial[2]
103         return
104
105     barco = barcos[indice]
106     id_barco = indice + 1 # Identificador unico para el barco.
107
108     if indice != 0 and sol_parcial[2] != barco and barcos[indice - 1] == barco:
109         """
110         Si el barco que agregamos por ultima vez es diferente al actual pero el
111         barco anterior tiene el mismo largo
112         que el actual, salteamos.
113         """
114         return batalla_naual_bt(tablero, barcos, demandas_filas, demandas_columnas,
115             indice + 1, sol_parcial, sol_optima)
116
117     cota_superior = sum(max(0, d - barco) for d in demandas_filas) + sum(max(0, d -
118         barco) for d in demandas_columnas)
119     if cota_superior >= sol_optima[0]:
120         return
121
122     if demandas_filas.sum() + demandas_columnas.sum() - (sum(barcos[indice:])*2) >=
123         sol_optima[0]:
124         """
125         Si las demandas pendientes menos la sumatoria de los largos de los barcos
126         que quedan multiplicada por 2
127         todavia no alcanza para ser menor a la solucion optima, descartamos esta
128         rama.
129         """
130         return
131
132     filas_prioritarias, columnas_prioritarias = obtener_filas_columnas_prioritarias
133         (demandas_filas, demandas_columnas)
134
135     # Probar todas las combinaciones de posicion y orientacion.
136     for fila in filas_prioritarias:
137         if demandas_filas[fila] == 0 or (barco > demandas_columnas[
138             columnas_prioritarias[0]] and barco > demandas_filas[fila]): break
139         for columna in columnas_prioritarias:
140             if demandas_columnas[columna] == 0 or (barco > demandas_filas[
141                 columna] and barco > demandas_filas[fila]): break
142             for orientacion in ['H', 'V']:

```

```
132         if puede_colocar_barco(tablero, barco, fila, columna, orientacion,
133                                demandas_filas, demandas_columnas):
134             # Colocar barco.
135             colocar_barco(tablero, barco, fila, columna, orientacion,
136                           id_barco, demandas_filas, demandas_columnas)
137
138             sol_parcial[0] = demandas_filas.sum() + demandas_columnas.sum()
139             sol_parcial[1] = tablero.copy()
140             sol_parcial[2] = barco
141
142             if sol_parcial[0] < sol_optima[0] or sol_parcial[0] - barcos
143                [-1]*2 >= 0:
144                 batalla_naval_bt(tablero, barcos, demandas_filas,
145                                 demandas_columnas, indice + 1, sol_parcial, sol_optima)
146             # Retirar barco.
147             colocar_barco(tablero, barco, fila, columna, orientacion, 0,
148                           demandas_filas, demandas_columnas)
149
150     return batalla_naval_bt(tablero, barcos, demandas_filas, demandas_columnas,
151                             indice + 1, sol_parcial, sol_optima)
```

Por la naturaleza de los algoritmos de backtracking, la complejidad temporal del algoritmo será exponencial. Como consecuencia de esto, el tiempo empirico necesario para resolver un problema de una complejidad grande, es demasiado grande como para ser considerado razonable. Para demostrar esto, debajo se muestran los test corridos con el algoritmo implementados, junto con el tiempo que conlleva resolver el problema que plantea dicho test.

Test	Tiempo empleado(segundos)
30'25'25	558.8437674045563
20'18'18	492.55675053596497
20'21'19	395.867155790329
20'13'15	212.61008429527283
15'20'20	51.86872625350952
20'17'15	2.9090425968170166

Como se puede ver, el tiempo necesario para resolver el problema se eleva mucho con una variación muy pequeña en la entrada. Para test que contemplan valores de una magnitud superior a los medidos, el tiempo necesario para resolver el problema asociado sería inabarcable para cumplir con alguna cota de tiempo de trabajo razonable. Por esto veremos que es muy útil tener una aproximación que tenga acotada su lejanía óptimo verdadero, puesto que estas aproximaciones requieren de tiempos mucho menores que los algoritmos exactos.

4. Modelo Programación Lineal

El algoritmo de programación lineal fue construido pensando en crear un tablero para cada barco, tablero representado por variables binarias, donde las variables binarias que conformaran el tablero de un barco específico solo valdrían "1" si el barco está ocupando la celda representada, además se usaron variables and's, como las vistas en las clases de programación lineal, para indicar que el barco solo podía estar colocado en el tablero si se encendían tantas variables binarias contiguas (ya sea en fila o en columna) como largo del tiene el barco, también se puso como condición que si se toma todo el sector alrededor de donde esta posicionado el barco (incluido donde está el barco), el resultado de la suma sea el largo del barco, esto se logró mediante la creación de variables and y or. Para que dos barcos no se puedan superponer en el tablero, para la copia del tablero armado por cada barco, se especificó que solo en una de las copias la casilla puede estar encendida, esto se hizo poniendo como condición que la suma por cada copia de la celda que exista debe dar menor o igual a 1. Además, para que las demandas no sean superadas, se especificó que la suma de las celdas de cada copia de una fila que exista, debía ser menor o igual que la demanda de la fila correspondiente, de manera análoga se realizó con las celdas. De esta manera se pensó el algoritmo de batalla naval.

Este en particular, nos costo mucho resolver de forma eficiente, la resolucion a los problemas por medio del algoritmo de Programacion Lineal planteado resuelve pero tarda un tiempo exagerado, al punto de tardar horas para la 3er prueba de la catedra que es la de 8.7_10. Al ser un punto como **opcional** en las consignas, por mas que no sea el mejor, nos parecia de buen aporte agregar lo planteado y que en base a ello recibir criticas constructivas.

```

1 def batalla_naval(tablero, barcos, demandas_filas, demandas_columnas):
2     problema = pulp.LpProblem("batalla_naval", pulp.LpMaximize)
3     variables_barcos = []
4     variables_barcos_posiciones = []
5     variables_celdas = []
6     for fila in tablero:
7         fila_variables = []
8         for celda in fila:
9             fila_variables.append(pulp.LpVariable("C(" + str(len(variables_celdas)) + ","
10                + str(len(fila_variables)) + ")", cat="Binary"))
11             variables_celdas.append(fila_variables)
12
13     for indice_barco in range(len(barcos)):
14         variables_barcos.append(pulp.LpVariable("B"+str(indice_barco), cat = "Binary"))
15
16     for indice_barco in range(len(barcos)):
17         barco_pos = []
18         for pos in range(len(tablero)*len(tablero[0])):
19             barco_pos.append(pulp.LpVariable("B" + str(indice_barco) + "P" + str(pos),
20                cat="Binary"))
21             variables_barcos_posiciones.append(barco_pos)
22
23     for pos in range(len(tablero)*len(tablero[0])):
24         problema += pulp.LpAffineExpression([(variables_barcos_posiciones[indice_barco]
25            ][pos], 1) for indice_barco in range(len(barcos))]) <= 1
26
27     for indice_barco in range(len(barcos)):
28         problema += pulp.LpAffineExpression([(variables_barcos_posiciones[indice_barco]
29            ][pos],1) for pos in range(len(tablero)*len(tablero[0]))]) <= barcos[
30            indice_barco]*variables_barcos[indice_barco]
31
32     for indice_barco in range(len(barcos)):
33         variables_and_filas = []
34         variables_and_columnas = []
35         or_columnas = pulp.LpVariable("SE_PONE_EN_FILA_BARCO_"+str(indice_barco), cat =
36            "Binary")
37         for fila in range(len(tablero)):
38             for comienzo in range(len(tablero[0]) - barcos[indice_barco]):
39                 variable_and = pulp.LpVariable("AND_COLUMNAS_ACTIVAS_DESDE_"+str(comienzo)+
40                    "_HASTA_"+str(comienzo+barcos[indice_barco]) + "_FILA_"+str(fila)+"BARCO_"+str(
41                    indice_barco), cat = "Binary")
42                 variable_and_borde_vacio = pulp.LpVariable("
43                    AND_BORDE_LIBRE_COLUMNAS_ACTIVAS_DESDE_"+str(comienzo)+"_HASTA_"+str(comienzo+
44                    barcos[indice_barco]) + "_FILA_"+str(fila)+"BARCO_"+str(indice_barco), cat = "
45                    Binary")
46                 for filab in range(max(fila - 1,0), min(fila + 2, len(tablero))):
47                     for columnab in range(max(0,comienzo - 1), min(len(tablero[0]),comienzo +
48                        barcos[indice_barco] + 1)):
49                         for indice_otro_barco in range(0, indice_barco):
50                             problema += variable_and_borde_vacio <= 1 -
51                             variables_barcos_posiciones[indice_otro_barco][len(tablero[0])*filab + columnab
52                                ]
53                             for indice_otro_barco in range(indice_barco + 1, len(barcos)):
54                                 problema += variable_and_borde_vacio <= 1 -
55                                 variables_barcos_posiciones[indice_otro_barco][len(tablero[0])*filab + columnab
56                                    ]
57                             problema += variable_and*barcos[indice_barco] <= pulp.LpAffineExpression([(
58                                variables_barcos_posiciones[indice_barco][len(tablero[0])*fila + columna],1)
59                                for columna in range(comienzo, comienzo + barcos[indice_barco])])
60                             problema += variable_and <= variable_and_borde_vacio
61                             variables_and_filas.append(variable_and)
62                             problema += pulp.LpAffineExpression([(var_and_fila, 1) for var_and_fila in
63                                variables_and_filas]) <= 1
64                             problema += or_columnas <= pulp.LpAffineExpression([(var_and_fila, 1) for

```

```

46     var_and_fila in variables_and_filas])
47     or_filas = pulp.LpVariable("SE_PONE_EN_COLUMNA_BARCO_"+str(indice_barco), cat =
48     "Binary")
49     for columna in range(len(tablero[0])):
50         for comienzo in range(len(tablero) - barcos[indice_barco]):
51             variable_and = pulp.LpVariable("AND_FILAS_ACTIVAS_DESDE_"+str(comienzo)+"
52             _HASTA_"+str(comienzo+barcos[indice_barco])+"_COLUMNA_"+str(columna)+"BARCO_"+
53             str(indice_barco), cat = "Binary")
54             variable_and_borde_vacio = pulp.LpVariable("
55             AND BORDE LIBRE FILAS ACTIVAS DESDE_"+str(comienzo)+"_HASTA_"+str(comienzo+
56             barcos[indice_barco]) + "_COLUMNA_"+str(columna)+"BARCO_"+str(indice_barco),
57             cat = "Binary")
58             for columnab in range(max(columna - 1,0), min(columna + 2, len(tablero[0]))
59             ):
60                 for filab in range(max(0, comienzo - 1),min(len(tablero) ,comienzo +
61                 barcos[indice_barco] + 1)):
62                     for indice_otro_barco in range(0, indice_barco):
63                         problema += variable_and_borde_vacio <= 1 -
64                         variables_barcos_posiciones[indice_otro_barco][len(tablero[0])*filab + columnab
65                         ]
66                         for indice_otro_barco in range(indice_barco + 1, len(barcos)):
67                             problema += variable_and_borde_vacio <= 1 -
68                             variables_barcos_posiciones[indice_otro_barco][len(tablero[0])*filab + columnab
69                             ]
70                             problema += variable_and*barcos[indice_barco] <= pulp.LpAffineExpression([(
71                             variables_barcos_posiciones[indice_barco][len(tablero[0])*fila + columna],1)
72                             for fila in range(comienzo, comienzo + barcos[indice_barco])])
73                             problema += variable_and <= variable_and_borde_vacio
74                             variables_and_columns.append(variable_and)
75                             problema += pulp.LpAffineExpression([(var_and_columna, 1) for var_and_columna
76                             in variables_and_columns]) <= 1
77                             problema += or_filas <= pulp.LpAffineExpression([(var_and_columna, 1) for
78                             var_and_columna in variables_and_columns])
79                             problema += or_filas + or_columns <= 1
80                             problema += variables_barcos[indice_barco] <= or_filas + or_columns
81
82     for fila in range(len(tablero)):
83         problema += pulp.LpAffineExpression([(variables_barcos_posiciones[indice_barco
84         ][len(tablero[0])*fila + columna], 1) for indice_barco in range(len(barcos))
85         for columna in range(len(tablero[0]))]) <= demandas_filas[fila]
86
87     for columna in range(len(tablero[0])):
88         problema += pulp.LpAffineExpression([(variables_barcos_posiciones[indice_barco
89         ][len(tablero[0])*fila + columna], 1) for indice_barco in range(len(barcos))
90         for fila in range(len(tablero))]) <= demandas_columnas[columna]
91
92     for fila in range(len(tablero)):
93         for columna in range(len(tablero[0])):
94             problema += variables_celdas[fila][columna] <= pulp.LpAffineExpression([(
95             variables_barcos_posiciones[pos_barco][len(tablero[0])*fila + columna], 1) for
96             pos_barco in range(len(barcos))])
97             problema += pulp.LpAffineExpression([(celda,2) for fila in variables_celdas for
98             celda in fila])
99     problema.solve()
100
101     for fila in range(len(tablero)):
102         for columna in range(len(tablero[fila])):
103             tablero[fila][columna] = sum([pulp.value(variables_barcos_posiciones[
104             indice_barco][len(tablero[0])*fila + columna]*(indice_barco + 1) for
105             indice_barco in range(len(barcos))])
106
107     return pulp.value(problema.objective),tablero

```

5. Aproximacion John Jellicoe

5.1. Codigo

El algoritmo de que propone John Jellicoe para encontrar una solución aproximada al resultado óptimo, consta de ir tratando de suplir la fila/columna con mayor demanda en cada iteracion, colocando el barco disponible mas grande que sea colocable en esa fila/columna, y repetir esto sucesivamente. El código implementado para lograr esto es el siguiente:

```
1 def batalla_naval(tablero, barcos, demandas_filas, demandas_columnas):
2     demandas_columnas_pos = [{"c", i, demandas_columnas[i]} for i in range(len(
3         demandas_columnas))]
4     demandas_filas_pos = [{"f", i, demandas_filas[i]} for i in range(len(
5         demandas_filas))]
6     demandas_totales = demandas_columnas_pos + demandas_filas_pos
7     barcos_aux = [(n, barcos[n]) for n in range(len(barcos))]
8     barcos_aux.sort(key=lambda numero_largo_barco: numero_largo_barco[1]*(-1))
9     while barcos_aux and existe_barco_que_entre_en_demanda(tablero,
10         demandas_filas_pos, demandas_columnas_pos, demandas_totales, barcos_aux):
11         info_demanda = max(demandas_totales, key=obtener_demanda)
12         tipo_demanda = info_demanda[0]
13         pos_demanda = info_demanda[1]
14         cantidad_demanda = info_demanda[2]
15         pos_barco = 0
16         posicion = encontrar_posicion(tablero, tipo_demanda, pos_demanda, barcos_aux[
17             pos_barco][1], demandas_filas_pos, demandas_columnas_pos)
18         while pos_barco < len(barcos_aux) and (barcos_aux[pos_barco][1] >
19             cantidad_demanda or posicion == -1):
20             pos_barco += 1
21         if pos_barco < len(barcos_aux):
22             posicion = encontrar_posicion(tablero, tipo_demanda, pos_demanda,
23                 barcos_aux[pos_barco][1], demandas_filas_pos, demandas_columnas_pos)
24         if pos_barco >= len(barcos_aux):
25             demandas_totales.remove(info_demanda)
26             continue
27         largo_barco = barcos_aux[pos_barco][1]
28         if tipo_demanda == "f":
29             for j in range(largo_barco):
30                 tablero[pos_demanda][posicion + j] = barcos_aux[pos_barco][0] + 1
31                 demandas_columnas_pos[posicion + j][2] -= 1
32             else:
33                 for j in range(largo_barco):
34                     tablero[posicion + j][pos_demanda] = barcos_aux[pos_barco][0] + 1
35                     demandas_filas_pos[posicion + j][2] -= 1
36                 barcos_aux.pop(pos_barco)
37                 info_demanda[2] -= largo_barco
38         return (sum(demandas_filas) + sum(demandas_columnas) -
39             devolver_demanda_total_cumplida(demandas_filas_pos) -
40             devolver_demanda_total_cumplida(demandas_columnas_pos)), tablero
41
42 def devolver_demanda_total_cumplida(demandas_totales):
43     demanda_cumplida = 0
44     for info_demanda in demandas_totales:
45         demanda_cumplida += info_demanda[2]
46     return demanda_cumplida
47
48 def existe_barco_que_entre_en_demanda(tablero, demandas_filas_pos,
49     demandas_columnas_pos, demandas_totales, barcos_aux):
50     demandas_totales_aux = demandas_totales.copy()
51     demandas_totales_aux.sort(key=obtener_demanda)
52     demandas_totales_aux.reverse()
53     largo_barco_mas_chico = barcos_aux[-1][1]
54     for demanda in demandas_totales_aux:
55         if demanda[0] == "f" and demandas_filas_pos[demanda[1]][2] <
56             largo_barco_mas_chico or demanda[0] == "c" and demandas_columnas_pos[demanda
```

```
[1]][2] < largo_barco_mas_chico:
51     return False
52     if encontrar_posicion(tablero, demanda[0], demanda[1], largo_barco_mas_chico,
53                             demandas_filas_pos, demandas_columnas_pos) != -1:
54         return True
55     return False
56 def encontrar_posicion(tablero, tipo_demanda, pos_demanda, largo_barco,
57                         demandas_filas_pos, demandas_columnas_pos):
58     espacios_libres_continuos = 0
59     if tipo_demanda == "f":
60         for columna in range(len(tablero[pos_demanda])):
61             if tablero[pos_demanda][columna] == 0 and demandas_columnas_pos[columna][2] >
62                 0:
63                 espacios_libres_continuos += 1
64             else:
65                 espacios_libres_continuos = 0
66                 if espacios_libres_continuos == largo_barco:
67                     siguiente_columna = columna + 1 if columna + 1 < len(tablero[pos_demanda])
68                     else columna
69                     columna_anterior = columna - largo_barco if columna - largo_barco >= 0 else
70                     columna + 1 - largo_barco
71                     fila_anterior = pos_demanda - 1 if pos_demanda - 1 >= 0 else pos_demanda +
72                     1
73                     fila_siguiente = pos_demanda + 1 if pos_demanda + 1 < len(tablero) else
74                     pos_demanda - 1
75                     fila_libre = True
76                     for col in range(columna_anterior, siguiente_columna + 1):
77                         if tablero[fila_anterior][col] != 0 or tablero[fila_siguiente][col] != 0:
78                             fila_libre = False
79                         if tablero[pos_demanda][siguiente_columna] == 0 and tablero[pos_demanda][
80                             columna_anterior] == 0 and fila_libre:
81                             return columna + 1 - largo_barco
82                             espacios_libres_continuos = 0
83     else:
84         for fila in range(len(tablero)):
85             if tablero[fila][pos_demanda] == 0 and demandas_filas_pos[fila][2] > 0:
86                 espacios_libres_continuos += 1
87             else:
88                 espacios_libres_continuos = 0
89                 if espacios_libres_continuos == largo_barco:
90                     siguiente_fila = fila + 1 if fila + 1 < len(tablero) else fila
91                     fila_anterior = fila - largo_barco if fila - largo_barco >= 0 else fila + 1
92                     - largo_barco
93                     columna_anterior = pos_demanda - 1 if pos_demanda - 1 >= 0 else pos_demanda
94                     + 1
95                     columna_siguiente = pos_demanda + 1 if pos_demanda + 1 < len(tablero[0])
96                     else pos_demanda - 1
97                     columna_libre = True
98                     for fil in range(fila_anterior, siguiente_fila + 1):
99                         if tablero[fil][columna_anterior] != 0 or tablero[fil][columna_siguiente]
100                             != 0:
101                             columna_libre = False
102                         if tablero[siguiente_fila][pos_demanda] == 0 and tablero[fila_anterior][
103                             pos_demanda] == 0 and columna_libre:
104                             return fila + 1 - largo_barco
105                             espacios_libres_continuos = 0
106     return -1
107 def obtener_demanda(info_demanda):
108     return info_demanda[2]
```

Si tratamos de conseguir una cota para la complejidad temporal del algoritmo, debemos ir analizando todas las partes que lo componen: Para analizar el código del algoritmo de batalla naval greedy, hay que primero tener en cuenta la complejidad de los algoritmos “existe_barco_que_entre_en_demanda” y “encontrar_posicion”. Se considerará que el tablero pasado por parámetro es de dimensión $n \times m$, donde n es la cantidad de filas y m la cantidad de columnas, la cantidad de barcos será b .

En cuanto al algoritmo de “encontrar posición”, se recorrerá cada columna o cada fila, según sea la naturaleza de la demanda que compete al momento de correr el algoritmo, y dentro de cada recorrido, se harán una cantidad finita de operaciones, acompañadas por un bucle for que recorre tanto como el largo del barco que compete posicionar, a lo sumo, esto será $O(\max(n,m))$. Por lo tanto el algoritmo será $O(\max(n,m) * \max(n,m))$, lo cual está acotado por $O((n+m)^2)$. Para existe barco que entre en demanda. Primero se copia todo el contenido de “demandas totales” a una variable auxiliar, “demandas totales” guarda un conjunto de información por cada demanda, sea tanto de fila como de columna, teniendo en cuenta que el tamaño del parámetro “demandas filas” es de tamaño n , y “demandas columnas” es de tamaño m , por lo tanto, la copia de “demandas totales” demora $O(n+m)$ en complejidad temporal. Luego de esto, se ordenará la lista copiada, lo que demorará $O((n+m) * \log(n+m))$, inmediatamente después se dará vuelta la lista, ocasionando una complejidad de $O(n+m)$. Una vez terminados los pasos anteriores, el algoritmo, entrará en un bucle for de $n+m$ repeticiones, poseyendo dentro un llamado a la función “encontrar posición”, de la cual ya se comentó su complejidad ($O((n+m)^2)$), quedando que la complejidad en esta parte del código será $O((n+m)^3)$, siendo esta última la mayor complejidad de todas y la complejidad de la función “existe barco que entre en demanda”. Pasando analizar el algoritmo de “batalla naval”, vemos que se guardaran con cierta disposición, en listas nuevas, cada uno de los elementos de la lista demandas filas (lo que nos da una complejidad de $O(n)$) y de la lista demandas columnas (lo que nos da una complejidad de $O(m)$), debajo de esto se realiza una unión de estas dos últimas, guardándose dentro de la variable “demandas totales” tal unión, y dando una complejidad de $O(n+m)$. Por consiguiente, se procede a realizar guardados con cierta disposición, en listas, para cada uno de los botes, dando una complejidad de $O(b)$, y se ordena, consiguiendo una complejidad de $O(b * \log(b))$. Como siguiente paso, se procederá a ingresar en un bucle while, el cual se repetirá mientras todavía queden barcos o mientras se puedan seguir satisfaciendo demandas, si la mayor demanda actual no puede ser satisfecha, se eliminará la mayor demanda y se procederá a la siguiente mayor demanda, probándose con cada barco en cada ocasión, en el peor de los casos, hasta que cada demanda llegue a estar satisfecha o llegue a un punto en el que ya no puede achicarse más, en el peor de los casos, se requerirán $(n*m)/2$ barcos para llenar todas las demandas, dando una cantidad de a lo sumo $(n*m)/2$ iteraciones del bucle while, además, por cada iteración, se usará la función “existe barco que entre en demanda”, la cual se sabe que tiene una complejidad de $O((n+m)^3)$. Ya dentro del código del bucle, se realizará una operación de búsqueda de máximo entre las demandas, lo que da una operación de $O(n+m)$, se realizará un llamado a la función “encontrar posición” (aportando esta, una complejidad de $O((n+m)^2)$), luego se realizarán iteraciones sobre un bucle while para buscar en qué posición puede estar el barco de máximo largo para la demanda seleccionada y se llamará la función “encontrar posición”, que como ya vimos, tiene complejidad $O((n+m)^2)$, esta se llamará a lo sumo por cada barco que tengamos, con lo que se aportará un $O(b * (n+m)^2)$ a la complejidad. En caso de que ningún barco se haya encontrado, se procederá a eliminar la demanda con “remove”, lo que proveerá una complejidad de $O(n+m)$, en caso de que el barco se hubiera hallado correctamente, se llamará a la función “encontrar posición” que sumará una complejidad de $O(n+m)$. Por último, se realizará una iteración por cada unidad de longitud del barco elegido (contribuyendo con a lo sumo $O(\max(n,m))$, los que se puede acotar por $O(n+m)$) y, se eliminará el barco usado, de las posibilidades a elegir, aportando un $O(b)$ a la complejidad. Para finalizar, se llama a la función “devolver demanda total cumplida”, que realiza una iteración por cada demanda, aportando un $O(n+m)$. Analizando, tenemos que la complejidad del bucle completo será de $O(n * m * b * (n+m)^2)$ al ser la complejidad más grande, esta será la complejidad del algoritmo.

5.2. Calidad de la Aproximación

Nuestro objetivo es el siguiente:

Sea I una instancia cualquiera del problema de La Batalla Naval, y $z(I)$ una solución óptima para dicha instancia, y sea $A(I)$ la solución aproximada, se define $A(I)/z(I) \leq r(A)$ para todas las instancias posibles. Calcular $r(A)$ para el algoritmo dado, demostrando que la cota está bien calculada.

Utilizando lo anterior podemos pensar, ¿cuál es una característica que diferencia a una solución

óptima de una aproximada, que podría ser peor? Nuestra respuesta es la demanda incumplida. Esta característica es clave para definir que tan buenas son nuestras soluciones, y así poder definir cual es la calidad de respuesta que nuestros algoritmos aproximados pueden brindarnos.

Con los test que la catedra nos proporcionó, y utilizando nuestro algoritmo de backtracking definido anteriormente para encontrar las demandas incumplidas optimas, realizamos divisiones con cada par de resultados (por un lado el del algoritmo aproximado y por otro el del algoritmo optimo). De estos son diez test de los que disponiamos, fueron cinco los casos en el que ambos algoritmos tuvieron el mismo resultado, es decir, el algoritmo aproximado encontro la solucion óptima. Y en cuanto a los casos en los que hubo diferencia, el calculo de $r(A)$ lo calculamos como (demanda incumplida aprox.)/(demanda incumplida opt.). Si tomamos el promedio de los valores, el resultado es 1.2, pero si miramos cada uno por separado, el peor resultado de todos es de 1.5. Esto nos da informacion sobre el valor de la cota para este algoritmo, primero viendo las veces que este algoritmo fue óptimo, lo cual habla positivamente de su desempeño, y en los casos en los que el algoritmo aproximado no fue óptimo, el valor de $r(A)$ se mantiene alrededor de un valor de 1.2, y en un solo caso, obtuvimos un valor de 1.5. Habiendo nombrado lo anterior, vamos a definir como la cota $r(A)$ para todas las instancias posibles con el valor 1.5. Dicho esto, también podemos decir que esta es una 1.5-aproximación.

Vamos a terminar de confirmar que esto es una 1.5 aproximación, haciendo test sobre sets de datos ya inmanejables por el algoritmo exacto, pero de los cuales sabemos cuales son las demandas incumplidas óptimas, y que puedan ser trabajados por el algoritmo aproximado.

Test	Demanda Total	Resultado óptimo	Resultado Greedy Jhon Jellicoe
500_500_500	140214	140214	138892
600_600_700	203432	203432	199222
1000_1000_1000	551288	541404	541164

Dados estos test, y sus demandas óptimas, como la demanda incumplida es de 0, no podemos calcular el error como en los casos anteriores (division por 0), así que para calcularlo, vamos a hacer el siguiente calculo:

$$(\text{Demanda incumplida aprox}) / (\text{Demanda total})$$

Estos calculos tienen un resultado promedio de 0.02, lo que nos dice que la solucion se aleja en alrededor de un 2 % de la solucion real, lo cual esta dentro de la cota que habiamos calculado antes. Con esto concluimos el analisis de este algoritmo y confirmamos que es una 1.5 aproximacion.

6. Aproximacion Greedy

El algoritmo greedy propuesto por el grupo consiste en ordenar las demandas de filas y columnas de mayor a menor, ordenar el largo de los barcos de la misma forma y para cada demanda colocar los barcos más grandes posibles hasta que ningun barco de los sobrantes pueda entrar. Luego se repite esto para la siguiente demanda. El código del mismo se puede encontrar debajo:

```
1 def batalla_naval(tablero, barcos, demandas_filas, demandas_columnas):
2     demandas_columnas_pos = [{"c", i, demandas_columnas[i]} for i in range(len(
3         demandas_columnas))]
4     demandas_filas_pos = [{"f", i, demandas_filas[i]} for i in range(len(
5         demandas_filas))]
6     demandas_totales = demandas_columnas_pos + demandas_filas_pos
7     demandas_totales.sort(key=lambda demanda: demanda[2]*(-1))
8     barcos_aux = [(n, barcos[n]) for n in range(len(barcos))]
9     barcos_aux.sort(key=lambda numero_largo_barco: numero_largo_barco[1]*(-1))
10    barcos_usados = {}
11    for demanda in demandas_totales:
12        for barco in barcos_aux:
13            if barco in barcos_usados or barco[1] > demanda[2]:
14                continue
```

```
13     posicion = encontrar_posicion(tablero, demanda[0], demanda[1], barco[1],
14     demandas_filas_pos, demandas_columnas_pos)
15     if posicion != -1:
16         demanda[2] -= barco[1]
17         for avance in range(posicion, posicion+barco[1]):
18             if demanda[0] == "f":
19                 demandas_columnas_pos[avance][2] -= 1
20                 tablero[demanda[1]][avance] = barco[0] + 1
21             else:
22                 demandas_filas_pos[avance][2] -= 1
23                 tablero[avance][demanda[1]] = barco[0] + 1
24                 barcos_usados[barco] = True
25     return (sum(demandas_filas) + sum(demandas_columnas) -
26     devolver_demanda_total_cumplida(demandas_totales)), tablero
27 def devolver_demanda_total_cumplida(demandas_totales):
28     demanda_cumplida = 0
29     for info_demanda in demandas_totales:
30         demanda_cumplida += info_demanda[2]
31     return demanda_cumplida
32
33 def encontrar_posicion(tablero, tipo_demanda, pos_demanda, largo_barco,
34     demandas_filas_pos, demandas_columnas_pos):
35     espacios_libres_continuos = 0
36     if tipo_demanda == "f":
37         for columna in range(len(tablero[pos_demanda])):
38             if tablero[pos_demanda][columna] == 0 and demandas_columnas_pos[columna][2] >
39             0:
40                 espacios_libres_continuos += 1
41             else:
42                 espacios_libres_continuos = 0
43             if espacios_libres_continuos == largo_barco:
44                 siguiente_columna = columna + 1 if columna + 1 < len(tablero[pos_demanda])
45                 else columna
46                 columna_anterior = columna - largo_barco if columna - largo_barco >= 0 else
47                 columna + 1 - largo_barco
48                 fila_anterior = pos_demanda - 1 if pos_demanda - 1 >= 0 else pos_demanda +
49                 1
50                 fila_siguiente = pos_demanda + 1 if pos_demanda + 1 < len(tablero) else
51                 pos_demanda - 1
52                 fila_libre = True
53                 for col in range(columna_anterior, siguiente_columna + 1):
54                     if tablero[fila_anterior][col] != 0 or tablero[fila_siguiente][col] != 0:
55                         fila_libre = False
56                     if tablero[pos_demanda][siguiente_columna] == 0 and tablero[pos_demanda][
57                     columna_anterior] == 0 and fila_libre:
58                         return columna + 1 - largo_barco
59                     espacios_libres_continuos = 0
60     else:
61         for fila in range(len(tablero)):
62             if tablero[fila][pos_demanda] == 0 and demandas_filas_pos[fila][2] > 0:
63                 espacios_libres_continuos += 1
64             else:
65                 espacios_libres_continuos = 0
66             if espacios_libres_continuos == largo_barco:
67                 siguiente_fila = fila + 1 if fila + 1 < len(tablero) else fila
68                 fila_anterior = fila - largo_barco if fila - largo_barco >= 0 else fila + 1
69                 columna_anterior = pos_demanda - 1 if pos_demanda - 1 >= 0 else pos_demanda
70                 + 1
71                 columna_siguiente = pos_demanda + 1 if pos_demanda + 1 < len(tablero[0])
72                 else pos_demanda - 1
73                 columna_libre = True
74                 for fil in range(fila_anterior, siguiente_fila + 1):
75                     if tablero[fil][columna_anterior] != 0 or tablero[fil][columna_siguiente]
76                     != 0:
77                         columna_libre = False
```

```

69     if tablero[siguiente_fila][pos_demanda] == 0 and tablero[fila_anterior][
pos_demanda] == 0 and columna_libre:
70         return fila + 1 - largo_barco
71         espacios_libres_continuos = 0
72     return -1
73
74 def obtener_demanda(info_demanda):
75     return info_demanda[2]

```

El algoritmo greedy propuesto, con la diferencia de que ordena las demandas (atribuyendo un $O(n+m)$ a la complejidad), tiene la misma complejidad que el algoritmo de la catedra hasta que se llega al bucle principal, a partir de ahí, hay que empezar a reanalizar. En este caso, el algoritmo propuesto, en lugar de utilizar un bucle while, utiliza un bucle for como bucle principal, este bucle realiza una iteración por cada demanda, realizando así $n + m$ iteraciones, además dentro de este mismo y fuera del for interno que se repite b veces en cada iteración del bucle principal, dentro de este bucle interno se llama a “encontrar posicion” (complejidad ya vista, siendo $O(n+m)^2$), finalmente, el último punto a destacar es el for interno al anterior, que realiza tantas iteraciones como unidades de largo tenga el barco que entro, esto es a lo sumo n o m según se coloque el barco alineado con las columnas o alineado con las filas, con lo que la complejidad será de $O(\max(n,m))$, esto puede ser acotado por $O(n+m)$. Analizando la complejidad del bucle principal, se llega a una complejidad de $O(b*(n + m)*(n + m)^2) = O(b*(n+m)^3)$. Por último, debajo de la finalización del bucle principal, se llamará a la función “sum” para las lista de demandas de filas y otra para lista de demandas de columnas, aportando una complejidad de $O(n + m)$ y se llamará a la función “devolver demanda total cumplida”, la cual aportara $O(n + m)$, por lo ya visto. Al analizar todas estas complejidades, se llega a la conclusión de que el algoritmo presenta una complejidad temporal de $O(b*(n+m)^3)$.

6.1. Comparación del algoritmo propuesto con otros implementados

Vamos a comparar los resultados obtenidos para las pruebas entregadas por la catedra, con los diferentes algoritmos de greedy y el resultado optimo que se llega a alcanzar por back tracking o programación lineal:

Test	Demanda Total	Resultado Óptimo	Resultado Greedy Jhon Jellicoe	Resultado Greedy Propuesto
3_3_2	11	4	4	4
5_5_6	18	12	12	12
8_7_10	53	26	26	26
10_3_3	14	6	6	6
10_10_10	40	40	34	32
12_12_21	58	46	40	34
15_10_15	67	40	40	40
20_20_20	120	104	104	88
20_25_30	247	172	152	120
30_25_25	360	202	164	144
500_500_500	140214	140214	138892	136614
600_600_700	203432	203432	199222	199330
1000_1000_1000	551288	551288	541404	541164

En general, se puede ver una tendencia a que el algoritmo de Jhon Jollicoe consigue soluciones más cercanas al óptimo global que el propuesto para este trabajo práctico, en la mayoría de casos, el algoritmo propuesto, o iguala la cercanía al óptimo de Jhon Jollicoe, o esta se aleja más del óptimo que este último, el único casos donde esto no se cumple es para el test de 600 filas, 600 columnas y 700 barcos (aún con esto, los resultados no difieren mucho entre ellos). Los últimos tres tests realizados son demasiado grandes como para ser resueltos en un tiempo razonable por los algoritmos que dan el mejor óptimo, sin embargo, incluso para problemas de tal magnitud, el algoritmo greedy sigue cumpliendo con su cota de error.

7. Conclusion

Un trabajo práctico muy variado por más que sea el mismo juego, teniendo diferentes objetivos, para unos cumplir estrictamente condiciones y para otros lograr un óptimo (de igual manera cumplir con condiciones pero no las mismas en su totalidad).

Exploramos por distintos algoritmos, al fin y al cabo, resolver un mismo problema. Analizar no solo distintas formas con una misma estrategia de llegar a un objetivo, sino que, por medio de distintas estrategias llegar a un objetivo, es una buena forma de no limitarse y saber que (casi) siempre hay alguna forma de llegar a algún objetivo. Es posible mezclar estrategias, por ejemplo, en el algoritmo de Backtracking y John Jellicoe utilizamos la idea de ordenar filas y columnas por mayor demanda, a su vez, que lo hacemos con los barcos de mayor a menor largo.

En fin, ampliamos la idea de que se puede llegar a un objetivo por medio de distintas estrategias, quizás no con los mismos resultados ni tampoco hablando en tiempos de ejecución.

8. Correcciones

En este apartado, se presentaran las correcciones hechas con respecto a la anterior entrega.

8.1. Correcciones Reduccion

Para finalizar por completo con la demostración, vamos a enunciar la reduccion completa:

Dado un conjunto de objetos $O = \{o_1, o_2, \dots, o_n\}$ con tamaños $s(o_i) \in \mathbb{N}$, un número K de contenedores disponibles y una capacidad fija C para cada contenedor, existe solucion para esta instancia de Bin Packing, si y solo si, existe una disposicion valida en el problema de la Batalla Naval, con un conjunto de barcos $A = \{a_1, a_2, \dots, a_n\}$ con tamaños $b(a_i) \in \mathbb{N}$ en un tablero de $2K - 1$ filas x $(C + \text{cantidad de barcos})$ columnas, y unas listas de restricciones F y G para filas y columnas respectivamente. Esto lo podemos afirmar porque la instancia de la Batalla Naval esta creada con el objetivo de imitar a la perfección el comportamiento de Bin Packing, el tablero simula ser los contenedores, tienen una capacidad que concuerda con la dada al principio para estos, y los barcos representan los objetos dados y sus respectivos tamaños. Además, las restricciones junto con las filas con capacidad 0 son las que efectivamente hacen que el tablero sea tratado como los propios contenedores.

Ahora la reducción para el otro lado:

Dado un conjunto de barcos $A = \{a_1, a_2, \dots, a_n\}$ con tamaños $b(a_i) \in \mathbb{N}$ en un tablero de K filas x C columnas, y una lista de restricciones F para filas y G para columnas, donde las restricciones tienen las características detalladas en las explicaciones anteriores, existe solucion para esta instancia de Batalla Naval, si y solo si, existe solucion para una instancia de Bin packing con un conjunto de objetos $O = \{o_1, o_2, \dots, o_n\}$ con tamaños $s(o_i) \in \mathbb{N}$, un número $(K + 1) / 2$ de contenedores disponibles y una capacidad fija $(C - \text{cantidad de barcos})$ para cada contenedor. Como la instancia de la Batalla Naval, es una que puede ser generada a partir del problema de Bin Packing, el tablero de la batalla naval puede ser representado perfectamente por contenedores, y estos tendrán la capacidad de las filas que corresponden a esos mismos contenedores. Por supuesto, los objetos van a ser tratados con la misma libertad que los barcos que representan en el tablero.

Con la demostracion enunciada en ambos sentidos, concluimos que el problema de la Batalla Naval, es NP-Completo.

8.2. Correcciones Backtracking

Realizando las modificaciones sugeridas, nos quedarían las funciones dentro del archivo de la siguiente forma:

"puede_colocar_barco"

- Utiliza funciones como *all()* y *flatten()* para verificar restricciones de adyacencia y demandas en filas y columnas, devolviendo True o False si es posible colocar o no. Estas funciones son esenciales para garantizar que las reglas del juego se cumplan antes de colocar un barco.

Evaluación de *all()*:

- **Función:** Verifica si todos los elementos en un iterable cumplen una condición.
- **Complejidad:** Su complejidad es $O(n)$, donde n es el número de elementos en el iterable.
- **Escenario:** En este caso, el tamaño del iterable es limitado por la longitud del barco o por las dimensiones locales del tablero (vecindad inmediata), lo que hace que el impacto de su uso sea mínimo.

Evaluación de *flatten()*:

- **Función:** Convierte una submatriz bidimensional en un arreglo unidimensional para facilitar la iteración.
- **Impacto:** Dado que se aplica a regiones pequeñas del tablero (área inmediata alrededor del barco), su sobrecarga computacional es baja.

Alternativas consideradas:

- **Evitar *flatten()*:** Podría iterarse directamente sobre índices bidimensionales, pero esto complicaría la lectura y mantenimiento del código.
- **Optimización de *all()*:** No se considera necesario, ya que los arreglos involucrados son pequeños y el impacto en el rendimiento global es insignificante.

En conclusión, el uso de *all()* y *flatten()* en este contexto es apropiado por su simplicidad y porque su impacto en el rendimiento global es limitado.

”colocar_barco”

- Coloca el barco en el tablero con la orientación que reciba, actualizando las demandas de las filas, columnas y la demanda incumplida para la solución. Todos estos, fueron pasados por parámetro.

”obtener_filas_columnas_prioritarias”

- En este algoritmo se utiliza **np.argsort** de la biblioteca **NumPy** para ordenar los índices de las filas y columnas en base a la demanda restante, de mayor a menor. Esta ordenación permite priorizar las posiciones más críticas en términos de demanda, reduciendo el espacio de búsqueda del backtracking. **Justificación del uso de *np.argsort*:**
 - **Eficiencia computacional:** *np.argsort* tiene una complejidad de $O(n \log n)$, lo que es manejable dado el tamaño esperado de las filas y columnas en el tablero de juego.
 - **Facilita la implementación:** Proporciona un método directo para obtener una lista ordenada de índices que puede ser recorrida de manera prioritaria.
 - **Alternativas consideradas:** Una alternativa sería mantener un ordenamiento manual durante el recorrido del algoritmo, pero esto agregaría complejidad innecesaria y afectaría la claridad del código. Otra opción sería usar estructuras como heaps, pero la implementación de éste es más compleja ya que deberíamos estar rehaciendo los heaps por cada modificación, por ejemplo, en el caso de agregar un barco de forma ”horizontal”, tenemos que rehacer el heap de columnas ya que las columnas encoladas tendrán distinta demanda. Haciendo de ésta última, una implementación un poco más efectiva pero muy compleja de implementar.

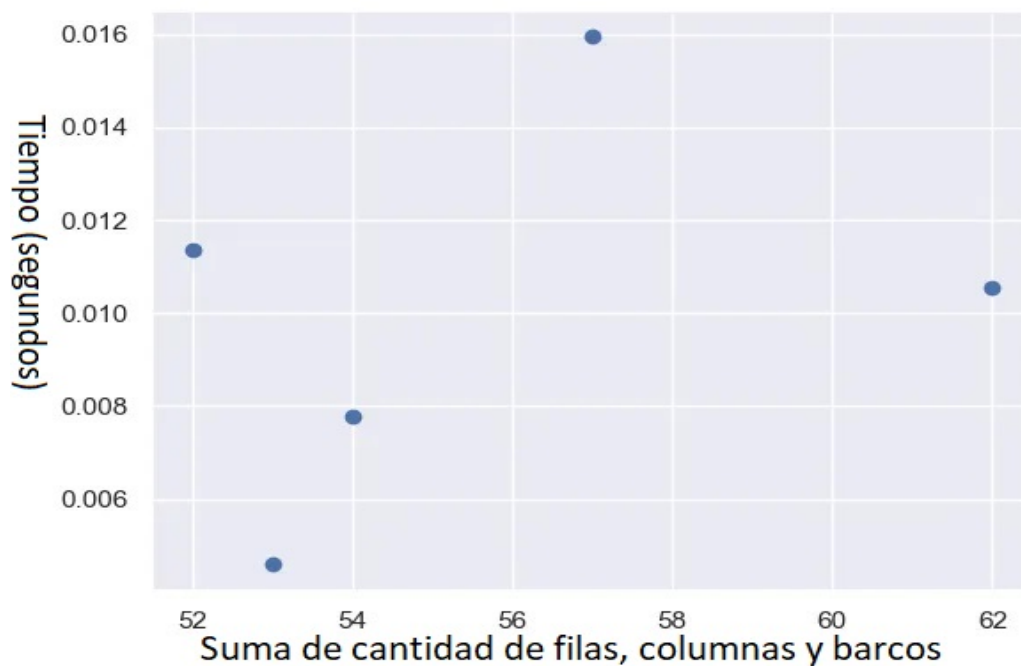
En resumen, **np.argsort** combina simplicidad, claridad y rendimiento, lo que lo convierte en una elección adecuada para esta parte del algoritmo.

"batalla_naval" y "batalla_naval_bt"

- Dentro de ambas funciones realizamos los cambios necesarios para no abusar de la función `sum()` la cual tiene una complejidad $O(n)$, pero hay casos en los que es inevitable usar.

Debajo se muestran mediciones de tiempo para diferentes pruebas, donde el algoritmo de back-tracking llegó al óptimo.

Test	Tiempo empleado(segundos)
17'16'20	0.004584741592407226
20'21'16	0.01594681739807129
20'21'21	0.01056375503540039
16'20'16	0.011361932754516602
19'17'18	0.007790803909301758



8.3. Correcciones Jhon Jellicoe

```
1 import heapq
2 import numpy as np
3 CANTIDAD_DEMANDA = 0
4 TIPO_DEMANDA = 1
5 POSICION_DEMANDA = 2
6 INDICE_BARCO = 0
7 LARGO_BARCO = 1
8 def batalla_naval(tablero, barcos, demandas_filas, demandas_columnas):
9     demandas_columnas_pos = [[demandas_columnas[i], "c", i] for i in range(len(
10         demandas_columnas))]
11     demandas_filas_pos = [[demandas_filas[i], "f", i] for i in range(len(
12         demandas_filas))]
13     demandas_totales = demandas_columnas_pos + demandas_filas_pos
14     heapq._heapify_max(demandas_totales)
15     barcos_aux = [(n, barcos[n]) for n in range(len(barcos))]
```

```

14 barcos_aux.sort(key=lambda numero_largo_barco: numero_largo_barco[1]*(-1))
15 indices_disponibles = {i: True for i in range(len(barcos_aux))}
16 while indices_disponibles and demandas_totales:
17     indice_barco = 0
18     info_demanda = heapq._heappop_max(demandas_totales)
19     tipo_demanda = info_demanda[TIPO_DEMANDA]
20     pos_demanda = info_demanda[POSICION_DEMANDA]
21     for indice_barco in indices_disponibles:
22         info_barco = barcos_aux[indice_barco]
23         barco = info_barco[LARGO_BARCO]
24         demandas = demandas_columnas_pos if tipo_demanda == "c" else
demandas_filas_pos
25         demandas_perpendiculares = demandas_filas_pos if tipo_demanda == "c" else
demandas_columnas_pos
26         celda = obtener_posicion(barco, tablero, pos_demanda, tipo_demanda, demandas,
demandas_perpendiculares)
27         if celda != -1:
28             fila_encontrada = pos_demanda if tipo_demanda == "f" else celda
29             columna_encontrada = pos_demanda if tipo_demanda == "c" else celda
30             colocar_barco(info_barco, tablero, tipo_demanda, fila_encontrada,
columna_encontrada)
31             info_demanda[CANTIDAD_DEMANDA] -= barco
32             reducir_demandas_perpendiculares(celda, barco, demandas_perpendiculares)
33             heapq.heappush(demandas_totales, info_demanda)
34             heapq._heapify_max(demandas_totales)
35             indices_disponibles.pop(indice_barco)
36             break
37
38 return (sum(demandas_filas) + sum(demandas_columnas) -
devolver_demanda_total_cumplida(demandas_filas_pos) -
devolver_demanda_total_cumplida(demandas_columnas_pos)), tablero
39
40 def obtener_posicion(barco, tablero, pos_demanda, tipo_demanda, demandas,
demandas_perpendiculares):
41
42     if demandas[pos_demanda][CANTIDAD_DEMANDA] < barco:
43         return -1
44     contiguos_disponibles = 0
45     posicion_contiguos = -1
46     for posicion_a_probar in range(len(demandas_perpendiculares)):
47         if posicion_contiguos + barco > len(demandas_perpendiculares):
48             return -1
49         posicion_fila_inicial = (posicion_a_probar if tipo_demanda == "c" else
pos_demanda) - 1
50         posicion_fila_final = (posicion_a_probar if tipo_demanda == "c" else
pos_demanda) + 2
51         posicion_columna_inicial = (pos_demanda if tipo_demanda == "c" else
posicion_a_probar) - 1
52         posicion_columna_final = (pos_demanda if tipo_demanda == "c" else
posicion_a_probar) + 2
53         limite_filas = len(demandas) if tipo_demanda == "f" else len(
demandas_perpendiculares)
54         limite_columnas = len(demandas_perpendiculares) if tipo_demanda == "f" else len
(demandas)
55         if np.all(tablero[max(posicion_fila_inicial, 0): min(posicion_fila_final,
limite_filas), max(posicion_columna_inicial, 0): min(posicion_columna_final,
limite_columnas)] == 0) and demandas_perpendiculares[posicion_a_probar][
CANTIDAD_DEMANDA] > 0:
56             if contiguos_disponibles == 0:
57                 posicion_contiguos = posicion_a_probar
58                 contiguos_disponibles += 1
59             else:
60                 posicion_contiguos = -1
61                 contiguos_disponibles = 0
62             if contiguos_disponibles == barco:
63                 return posicion_contiguos
64     return -1
65
66 def colocar_barco(info_barco, tablero, tipo_demanda, fila_a_probar,
columna_a_probar):

```

```

67 posicion_a_probar = fila_a_probar if tipo_demanda == "c" else columna_a_probar
68 for posicion in range(posicion_a_probar, posicion_a_probar + info_barco[
    LARGO_BARCO]):
69     fila = fila_a_probar if tipo_demanda == "f" else posicion
70     columna = columna_a_probar if tipo_demanda == "c" else posicion
71     tablero[fila][columna] = info_barco[INDICE_BARCO] + 1
72
73
74 def reducir_demandas_perpendiculares(posicion_perpendicular, barco,
    demandas_perpendiculares):
75     for posicion in range(posicion_perpendicular, posicion_perpendicular + barco):
76         demandas_perpendiculares[posicion][CANTIDAD_DEMANDA] -= 1
77
78 def devolver_demanda_total_cumplida(demandas_totales):
79     demanda_cumplida = 0
80     for info_demanda in demandas_totales:
81         demanda_cumplida += info_demanda[CANTIDAD_DEMANDA]
82     return demanda_cumplida

```

Si tratamos de conseguir una cota para la complejidad temporal del algoritmo, debemos ir analizando todas las partes que lo componen:

Para analizar el código del algoritmo de batalla naval greedy, hay que primero tener en cuenta la complejidad de los algoritmos “**obtener_posicion**”, “**colocar_barco**”, “**reducir_demandas_perpendiculares**” y “**devolver_demanda_total_cumplida**”. Se considerará que el tablero pasado por parámetro es de dimensión $n \times m$, donde n es la cantidad de filas y m la cantidad de columnas, la cantidad de barcos será b .

En cuanto al algoritmo de “**obtener_posicion**”, se realizarán una serie de comprobaciones de complejidad $O(1)$ con condicionales, y luego se pasa a realizar un bucle *for* por cada celda disponible para la fila/columna de la demanda correspondiente, dentro de este bucle se realizará una comprobación sobre el tablero, por cada celda posible, hasta terminar las celdas disponibles en la fila/columna de la demanda correspondiente, o hasta que se consigan tantas celdas contiguas disponibles en la fila/columna correspondiente como largo tenga el barco (esta corroboración se hace en $O(9) = O(1)$, ya que corrobora que el sector alrededor de la celda correspondiente este disponible en su totalidad), esto provocará que encontrar la posición del barco sea $O(\max(n, m))$. Analizando la función “**colocar_barco**”, vemos que se realiza una iteración por cada unidad de largo que tenga el barco, realizando una cantidad de operaciones finitas en cada iteración, lo que proporciona una complejidad de $O(\max(n, m))$ por lo ya mencionado. En el caso de la función “**reducir_demandas_perpendiculares**”, podemos ver que también se itera por cada unidad de largo del barco colocado, y de nuevo, se realizan una cantidad finita de operaciones, lo que nos lleva a una complejidad similar a la anterior $O(\max(n, m))$. Por último, si analizamos “**devolver_demanda_total_cumplida**”, vemos que se itera por las demandas de las filas y las columnas en su totalidad, realizando operaciones finitas en cada iteración, aportando una complejidad de $O(n + m)$.

Pasando a analizar el algoritmo de “**batalla_naval**”, vemos que se guardarán, con cierta disposición en listas nuevas, cada uno de los elementos de la lista *demandas_filas* (lo que nos da una complejidad de $O(n)$) y de la lista *demandas_columnas* (lo que nos da una complejidad de $O(m)$). Debajo de esto, se realiza una unión de estas dos últimas, guardándose dentro de la variable *demandas_totales* tal unión, y dando una complejidad de $O(n + m)$. Luego, se realiza *heapify* de *demandas_totales*, consiguiendo una complejidad de $O(n + m)$. Por consiguiente, se procede a realizar guardados, con cierta disposición, en listas, para cada uno de los barcos, dando una complejidad de $O(b)$, y se ordena, consiguiendo una complejidad de $O(b \log b)$. Finalmente, se creará un diccionario que contenga los índices de los barcos que todavía no han sido utilizados, aportando $O(b)$ a la complejidad. Como siguiente paso, se procederá a ingresar en un bucle *while*, el cual se repetirá mientras todavía queden barcos o existan demandas por satisfacer. En el peor caso, se colocarán barcos hasta completar $b - 1$ barcos, sin eliminar ninguna demanda en el proceso, y luego se intentará ese barco en las $n + m$ demandas, sin tener éxito. Esto provocaría una complejidad de $O(n + m + b)$. Dentro de este bucle, se realizarán primero una serie de operaciones $O(1)$ y un desencolamiento desde un *heap*, el cual aportará $O(\log(n + m))$ a la complejidad temporal, hasta encontrarse con otro bucle que itera por cada barco no usado, intentando poner este en la

fila o columna correspondiente a la demanda desencolada. Ya dentro de este último **for**, se realizarán ciertas operaciones de tiempo constante, para luego llamar a la función **obtener_posicion**, aportando $O(\max(n, m))$, como ya se mencionó. En caso de que la función **obtener_posicion** devuelva una posición válida, se colocará el barco en el tablero, en la posición obtenida, con la función **colocar_barco**, lo que tomará un tiempo de $O(\max(n, m))$. Luego se llamará a la función **reducir_demandas_perpendiculares**, lo que aportará $O(\max(n, m))$ en complejidad. Se realizará un **heappush** con los datos actualizados de la demanda desencolada, lo que acarreará otro aumento de la complejidad en $O(\log(n + m))$. Luego se hará **heapify** de la lista resultante, conllevando un incremento de $O(n + m)$ temporalmente. Estos últimos pasos, luego de corroborar que la posición devuelta sea válida, solo se harán **una vez por demanda**, por lo que el código desde **colocar_barco** solo se ejecutaría, como mucho, una vez por iteración del bucle **while** principal. Finalmente, la complejidad total del algoritmo sería:

$$\begin{aligned} O(n) + O(m) + O(n + m) + O(n + m) + O(b) + O(b \log b) \\ + O(b) + O((n + m + b) \log(n + m)) \\ + O((n + m + b)b \max(n, m)) \\ + O((n + m + b)(\max(n, m) + \max(n, m) \\ + \max(n, m) + \max(n, m) + \log(n + m) + (n + m))). \end{aligned}$$

Si la matriz es cuadrada y $b < 2n$, entonces la complejidad será $O(b \cdot n^2)$. En caso de que $2n < b$, la complejidad quedaría como $O(n \cdot b^2)$.

8.4. Corrección Calidad de la Aproximación

Nuestro objetivo es el siguiente:

Sea I una instancia cualquiera del problema de La Batalla Naval, y $z(I)$ una solución óptima para dicha instancia, y sea $A(I)$ la solución aproximada, se define $A(I)/z(I) \leq r(A)$ para todas las instancias posibles. Calcular $r(A)$ para el algoritmo dado, demostrando que la cota está bien calculada.

Utilizando lo anterior podemos pensar, ¿cuál es una característica que diferencia a una solución óptima de una aproximada, que podría ser peor? Nuestra respuesta es la demanda incumplida. Esta característica es clave para definir que tan buenas son nuestras soluciones, y así poder definir cual es la calidad de respuesta que nuestros algoritmos aproximados pueden brindarnos.

Con los test que la cátedra nos proporcionó, y utilizando nuestro algoritmo de backtracking definido anteriormente para encontrar las demandas incumplidas óptimas, realizamos divisiones con cada par de resultados (por un lado el del algoritmo aproximado y por otro el del algoritmo óptimo). De estos son diez test de los que disponíamos, fueron cinco los casos en el que ambos algoritmos tuvieron el mismo resultado, es decir, el algoritmo aproximado encontró la solución óptima. Y en cuanto a los casos en los que hubo diferencia, el cálculo de $r(A)$ lo calculamos como (demanda incumplida aprox.)/(demanda incumplida opt.). Si tomamos el promedio de los valores, el resultado es 1.5, pero si miramos cada uno por separado, el peor resultado de todos es de 2. Esto nos da información sobre el valor de la cota para este algoritmo, primero viendo las veces que este algoritmo fue óptimo, lo cual habla positivamente de su desempeño, y en los casos en los que el algoritmo aproximado no fue óptimo, el valor de $r(A)$ se mantiene alrededor de un valor de 1.5, y en un solo caso, obtuvimos un valor de 2. Habiendo nombrado lo anterior, vamos a definir como la cota $r(A)$ para todas las instancias posibles con el valor 2. Dicho esto, también podemos decir que esta es una 2-aproximación.

Vamos a terminar de confirmar que esto es una 2-aproximación, haciendo test sobre sets de datos ya inmanejables por el algoritmo exacto, pero de los cuales sabemos cuales son las demandas incumplidas óptimas, y que puedan ser trabajados por el algoritmo aproximado.

Test	Demanda Total	Resultado óptimo	Resultado Greedy Jhon Jellicoe
500_500_500	180404	180404	176072
600_600_700	272844	272844	265650
1000_1000_1000	746332	746332	718908

Dados estos test, y sus demandas óptimas, como la demanda incumplida es de 0, no podemos calcular el error como en los casos anteriores (division por 0), así que para calcularlo, vamos a hacer el siguiente calculo:

$$(\text{Demanda incumplida aprox}) / (\text{Demanda total})$$

Estos calculos tienen un resultado promedio de 0.03, lo que nos dice que la solucion se aleja en alrededor de un 3% de la solucion real, lo cual esta dentro de la cota que habiamos calculado antes. Con esto concluimos el analisis de este algoritmo y confirmamos que es una 2-aproximacion.

8.5. Correcciones Aproximación Greedy

```
1 import numpy as np
2 def batalla_naual(tablero, barcos, demandas_filas, demandas_columnas):
3     demandas_columnas_pos = [{"c", i, demandas_columnas[i]} for i in range(len(
4         demandas_columnas))]
5     demandas_filas_pos = [{"f", i, demandas_filas[i]} for i in range(len(
6         demandas_filas))]
7     demandas_totales = demandas_columnas_pos + demandas_filas_pos
8     demandas_totales.sort(key=lambda demanda: demanda[2]*(-1))
9     barcos_aux = [(n, barcos[n]) for n in range(len(barcos))]
10    barcos_aux.sort(key=lambda numero_largo_barco: numero_largo_barco[1]*(-1))
11    barcos_usados = {}
12    for demanda in demandas_totales:
13        for barco in barcos_aux:
14            if barco in barcos_usados or barco[1] > demanda[2]:
15                continue
16            demandas = demandas_filas_pos if demanda[0] == "f" else demandas_columnas_pos
17            demandas_perpendiculares = demandas_columnas_pos if demanda[0] == "f" else
18            demandas_filas_pos
19            posicion_posible = obtener_posicion(barco[1], tablero, demanda[1], demanda
20            [0], demandas, demandas_perpendiculares)
21            if posicion_posible != -1:
22                fila_encontrada = demanda[1] if demanda[0] == "f" else posicion_posible
23                columna_encontrada = demanda[1] if demanda[0] == "c" else posicion_posible
24                colocar_barco(barco, tablero, demanda[0], fila_encontrada,
25                columna_encontrada)
26                demanda[2] -= barco[1]
27                reducir_demandas_perpendiculares(posicion_posible, barco[1],
28                demandas_perpendiculares)
29                barcos_usados[barco] = True
30
31    return (sum(demandas_filas) + sum(demandas_columnas) -
32            devolver_demanda_total_cumplida(demandas_totales)), tablero
33
34 def devolver_demanda_total_cumplida(demandas_totales):
35     demanda_cumplida = 0
36     for info_demanda in demandas_totales:
37         demanda_cumplida += info_demanda[2]
38     return demanda_cumplida
39
40 def obtener_posicion(barco, tablero, pos_demanda, tipo_demanda, demandas,
41     demandas_perpendiculares):
42
43     if demandas[pos_demanda][2] < barco:
44         return -1
45     contiguos_disponibles = 0
46     posicion_contiguos = -1
47     for posicion_a_probar in range(len(demandas_perpendiculares)):
48         if posicion_contiguos + barco > len(demandas_perpendiculares):
```

```

41     return -1
42     posicion_fila_inicial = (posicion_a_probar if tipo_demanda == "c" else
43     pos_demanda) - 1
43     posicion_fila_final = (posicion_a_probar if tipo_demanda == "c" else
44     pos_demanda) + 2
44     posicion_columna_inicial = (pos_demanda if tipo_demanda == "c" else
45     posicion_a_probar) - 1
45     posicion_columna_final = (pos_demanda if tipo_demanda == "c" else
46     posicion_a_probar) + 2
46     limite_filas = len(demandas) if tipo_demanda == "f" else len(
47     demandas_perpendiculares)
47     limite_columnas = len(demandas_perpendiculares) if tipo_demanda == "f" else len(
48     demandas)
48     if np.all(tablero[max(posicion_fila_inicial, 0): min(posicion_fila_final,
49     limite_filas), max(posicion_columna_inicial, 0): min(posicion_columna_final,
50     limite_columnas)] == 0) and demandas_perpendiculares[posicion_a_probar][2] > 0:
51         if contiguos_disponibles == 0:
52             posicion_contiguos = posicion_a_probar
53             contiguos_disponibles += 1
54         else:
55             posicion_contiguos = -1
56             contiguos_disponibles = 0
57         if contiguos_disponibles == barco:
58             return posicion_contiguos
59     return -1
60
61 def colocar_barco(info_barco, tablero, tipo_demanda, fila_a_probar,
62     columna_a_probar):
63     posicion_a_probar = fila_a_probar if tipo_demanda == "c" else columna_a_probar
64     for posicion in range(posicion_a_probar, posicion_a_probar + info_barco[1]):
65         fila = fila_a_probar if tipo_demanda == "f" else posicion
66         columna = columna_a_probar if tipo_demanda == "c" else posicion
67         tablero[fila][columna] = info_barco[0] + 1
68
69 def reducir_demandas_perpendiculares(posicion_perpendicular, barco,
70     demandas_perpendiculares):
71     for posicion in range(posicion_perpendicular, posicion_perpendicular + barco):
72         demandas_perpendiculares[posicion][2] -= 1

```

El algoritmo greedy propuesto, con la diferencia de que ordena las demandas (atribuyendo un $O(n + m)$ a la complejidad), tiene la misma complejidad que el algoritmo de la cátedra hasta que se llega al bucle principal. A partir de ahí, hay que empezar a reanalizar. En este caso, el algoritmo propuesto, en lugar de utilizar un bucle **while**, utiliza un bucle **for** como bucle principal. Este bucle realiza una iteración por cada demanda, realizando así $n + m$ iteraciones. Además, dentro de este mismo, se realizará otro bucle **for** que iterará por cada barco. Dentro de este bucle interno, las únicas acciones que no se realizan en tiempo constante son las de las funciones **encontrar_posicion**, **colocar_barco** y **reducir_demandas_perpendiculares**, con las complejidades ya analizadas de $O(\max(n, m))$ para las tres. Por último, debajo de la finalización del bucle principal, se llamará a la función **sum** para la lista de demandas de filas y otra para la lista de demandas de columnas, aportando una complejidad de $O(n + m)$. También se llamará a la función **devolver_demanda_total_cumplida**, la cual aportará $O(n + m)$, como ya se vio.

Al analizar todas estas complejidades, se llega a la conclusión de que el algoritmo presenta una complejidad temporal de $O(b \cdot (n + m)^2)$. Si la matriz es cuadrada, esto se traduce a $O(b \cdot n^2)$.

8.6. Correcciones Comparación del algoritmo propuesto con otros implementados

Vamos a comparar los resultados obtenidos para las pruebas entregadas por la cátedra, con los diferentes algoritmos de greedy y el resultado óptimo que se llega a alcanzar por back tracking o programación lineal:

Test	Demanda Total	Resultado Óptimo	Resultado Greedy Jhon Jellicoe	Resultado Greedy Propuesto
3.3.2	11	4	4	4
5.5.6	18	12	12	12
8.7.10	53	26	26	26
10.3.3	14	6	6	6
10.10.10	40	40	36	32
12.12.21	58	46	40	34
15.10.15	67	40	40	40
20.20.20	120	104	88	88
20.25.30	247	172	152	146
30.25.25	360	202	170	164
500.500.500	180404	180404	176072	175538
600.600.700	272844	272844	265650	265724
1000.1000.1000	746332	746332	718908	719576

En general, se puede ver una tendencia a que el algoritmo de Jhon Jollicoe consigue soluciones más cercanas al óptimo global que el propuesto para este trabajo práctico, en la mayoría de casos, el algoritmo propuesto, o iguala la cercanía al óptimo de Jhon Jollicoe, o esta se aleja más del óptimo que este último, los únicos casos donde esto no se cumple es para el test de 600 filas, 600 columnas y 700 barcos, y el de 1000 filas, 1000 columnas y 1000 barcos (aún con esto, los resultados no difieren mucho entre ellos). Los últimos tres tests realizados son demasiado grandes como para ser resueltos en un tiempo razonable por los algoritmos que dan el mejor óptimo, sin embargo, incluso para problemas de tal magnitud, el algoritmo greedy sigue cumpliendo con su cota de error.